

7.4 - Fallback e degradação graciosa

Como manter o sistema de pé quando uma parte cai

Falhar inteiro nem sempre é necessário

Duas estratégias para perda parcial

- **Fallback responde mesmo com a dependência fora**
 - usa cache, valor padrão ou funcionalidade reduzida
- **Graceful degradation reduz escopo de forma controlada**
 - desliga o que é secundário e preserva o essencial
- **A degradação aceitável é decidida no design**
 - você escolhe antes o que pode cair sem quebrar o negócio
- **O usuário recebe menos, mas continua atendido**
 - uma mensagem clara evita a sensação de sistema quebrado

Maps instável, 80% do serviço de pé

- **API do Google Maps fica instável por 45 minutos**
 - ocorre numa tarde de pico de roteirização

LUNALOG

A API do Google Maps ficou instável por 45 minutos numa tarde de pico. Como a LunaLog tinha fallback implementado, o sistema serviu rotas calculadas do cache Redis para as 200 principais rotas urbanas e desabilitou temporariamente a roteirização dinâmica nas cidades menores, com uma mensagem clara para o usuário. O resultado foi 80% da funcionalidade mantida e zero downtime. Sem esse fallback, 100% do serviço de rotas estaria fora durante toda a instabilidade do Maps.

Fallback e degradação não são a mesma coisa

Fallback gracioso

- Substitui a resposta de uma dependência morta
- Serve cache, padrão ou versão simplificada
- Atua no ponto específico da chamada
- Usuário muitas vezes nem percebe

Graceful degradation

- Reduz o escopo do sistema como um todo
- Desliga recursos secundários sob sobrecarga
- Atua no nível da experiência completa
- Usuário percebe, mas continua operando

Degradação só funciona se foi projetada

- **Decida no design o que pode cair primeiro**
- **Defina a resposta de fallback de cada dependência**

Fallback e degradação não acontecem por acaso, eles são projetados antes do incidente. Liste cada dependência externa, defina qual é a resposta aceitável quando ela falhar, e decida quais recursos podem ser sacrificados para manter o núcleo funcionando. O que não foi modelado vira queda total.

- A LunaLog manteve 80% no ar porque planejou o cenário antes



Você projetou o fallback. Definiu a degradação aceitável. Está tudo no diagrama. Mas tem uma pergunta incômoda: isso funciona de verdade em produção, ou só na sua cabeça e no ambiente de teste? A maioria dos planos de resiliência nunca foi acionada para valer. E existe uma prática que injeta falha de propósito, em produção, para descobrir o que o sistema realmente faz.

Chaos engineering: testar resiliência de propósito